

Department of Computer Science and Engineering

Indian Institute of Technology Guwahati

Lab 1

Processes, Pipes, and the Luit System-Call Boundary

CS3106L - Operating Systems: Concepts and Construction

BrahmaputraOS / kernel *Luit* - Lab 1 Student Release

Student handout v1.3 - student distribution

Graded assignment

User-space programming, generated-ABI inspection, and guided GDB tracing

Do not modify the kernel in this lab.

Dr. Satyajit Das

Department of Computer Science and Engineering, IIT Guwahati

Contents

1 Purpose of Lab 1	3
1.1 Learning outcomes	3
1.2 Working with the supplied skeleton files (IMPORTANT)	3
2 The Lab 1 Release Contract	4
2.1 Use the exact student starter	4
2.2 Establish a clean baseline	4
2.3 Files you must read before implementation	5
3 Task 1 - Implement sleep	5
3.1 Required behaviour	6
3.2 Manual tests	6
4 Task 2 - Implement pingpong	7
4.1 Descriptor ownership	7
4.2 Correctness requirements	8
4.3 Required tests	8
5 Task 3 - Implement pstree	9
5.1 The interface supplied by the release	9
5.2 Command-line and output contract	9
5.3 Algorithmic requirements	9
5.4 Required tests	10
6 Task 4 - Implement tools/syscallmap.py	11
6.1 The generated ABI pipeline	11
6.2 Required invocation and output	11
6.3 Required validations	12
6.4 Fault testing your checker	12
7 Guided GDB Exercise - Trace sleep 5	13
7.1 Inspect the generated user stub statically	13
7.2 Start the two-terminal debugging session	13
7.3 Follow the same process across the boundary	14
8 Integration and Testing Requirements	16
8.1 Required source-tree changes	16
8.2 Build and regression sequence	16

8.3	Minimum test matrix	16
8.4	What grading tests will examine	17
9	Submission Through the OneDrive File-Request Link	17
9.1	Complete the identity and evidence files	17
9.2	Create the roll-numbered ZIP automatically	17
9.3	Upload procedure	18
9.4	Final submission checklist	18
10	Assessment Rubric	19
11	AI/LLM Use and Academic Integrity	19
12	Getting Unstuck	20
13	Viva Preparation	20
14	The Main Ideas to Carry Forward	21

1 Purpose of Lab 1

In Lab 0 you built and explored a complete Luit system. In this assignment you will use the operating-system interface as a programmer, inspect a live process snapshot supplied by the kernel, verify how Luit generates its system-call ABI, and follow one system call through GDB.

arguments → **system calls** → **processes** → **file descriptors** → **pipes** → **traps**

The assignment deliberately combines familiar Unix mechanisms with two Luit-specific tasks: constructing a process tree from `procstat()` and checking the generated ABI derived from `kernel/syscall.tbl`. The goal is not merely to obtain the expected output. You must be able to explain why every process exists, which file descriptors it owns, and which RISC-V registers carry a system call across the user/kernel boundary.

1.1 Learning outcomes

After completing this lab, you should be able to:

1. write and integrate a new Luit user program;
2. validate command-line arguments without relying on the host C library;
3. use `fork()`, `pipe()`, `read()`, `write()`, `close()`, and `wait()` correctly;
4. explain file-descriptor inheritance across `fork()`;
5. explain why a pipe reader observes end-of-file only after every write descriptor is closed;
6. interpret a one-time `procstat()` snapshot and construct a deterministic process tree;
7. explain how `kernel/syscall.tbl` generates kernel numbers, the dispatch table, names, and user stubs;
8. inspect the RISC-V system-call ABI using disassembly and GDB; and
9. preserve the baseline behaviour of the complete operating system.

1.2 Working with the supplied skeleton files (IMPORTANT)

The starter repository already contains the required source files and already includes the three guest programs in `UPROGS`. You do **not** create new files and you do **not** edit the Makefile. Complete the marked regions in these four files:

File	TODO labels	Main responsibility
<code>user/sleep.c</code>	S1–S3	validated decimal conversion and one system call
<code>user/pingpong.c</code>	P1–P4	two-pipe process protocol and descriptor cleanup
<code>user/pstree.c</code>	T1–T7	one snapshot, sorting, traversal, and consistency checks
<code>tools/syscallmap.py</code>	A1–A6	parsing and cross-checking the generated ABI

Each editable region is delimited by a numbered pair such as:

```
/* TODO-BEGIN P3: implement the child side of the protocol. */  
... your implementation ...  
/* TODO-END P3 */
```

Keep the begin/end markers so that you and the TAs can locate your changes. You may add helper functions, constants, and local data structures inside the same four files. Do not alter supplied function interfaces unless the handout explicitly permits it. The starter code is intentionally compilable but prints a clear “TODO not implemented” error until the central logic is completed.

To list all work regions:

LINUX HOST TERMINAL - run outside QEMU

```
grep -RIn 'TODO-BEGIN\|TODO-END' \  
  user/sleep.c user/pingpong.c user/pstree.c tools/syscallmap.py
```

Skeletons are guidance, not solutions

The framework removes repository setup errors but does not implement the algorithms. Replacing a complete skeleton with a public xv6 solution, deleting its contract checks, or using an agent to fill all TODOs is not permitted.

2 The Lab 1 Release Contract

2.1 Use the exact student starter

Extract the archive supplied for your laboratory section. From the repository root, verify the release:

LINUX HOST TERMINAL - run outside QEMU

```
cat LAB01_RELEASE.txt  
ls user/sleep.c user/pingpong.c user/pstree.c  
ls tools/syscallmap.py tools/package_lab01.py  
sed -n '1,80p' kernel/pstat.h  
make lab01-check
```

The Lab 1 version of `kernel/pstat.h` must expose:

```
struct pstat {  
    int  pid;  
    int  ppid;  
    int  state;  
    char name[16];  
};
```

The release already implements kernel-side `ppid` support and already lists `sleep`, `pingpong`, and `pstree` in `UPROGS`. Read those supplied changes, but do not modify them.

A required file or `ppid` is missing

Stop and contact a TA. Do not repair the release by copying files from another student or by adding a kernel implementation yourself.

2.2 Establish a clean baseline

Before editing a TODO region, run:

LINUX HOST TERMINAL - run outside QEMU

```
make clean
make
make lab01-check
make grade
```

`make` must build the complete starter, including the intentionally incomplete Lab 1 programs. `make lab01-check` checks the release structure and protected files. `make grade` runs the existing Luit regression suite; it does not test your new Lab 1 behaviour, but it must pass before and after your work.

Checkpoint: Baseline recorded

Save the final clean-build line, the `lab01-check` result, and the public regression summary. If the untouched starter fails, show the complete output to a TA before changing source code.

2.3 Files you must read before implementation

Read targeted portions rather than every line of the repository:

Path	Question to answer while reading
<code>user/echo.c</code> , <code>user/kill.c</code>	What is the normal structure of a small Luit user program?
<code>user/ulib.h</code> , <code>user/ulib.c</code>	Which calls are system calls, and which are user-library functions?
<code>user/sh.c</code> , especially PIPE	How does the shell connect descriptors and guarantee pipe EOF?
<code>user/ps.c</code>	How is a <code>procstat()</code> snapshot requested and displayed?
<code>kernel/pstat.h</code>	Which process fields are visible to user space?
<code>kernel/proc.c</code> , <code>procstat()</code>	Why is each process slot copied while its lock is held, but <code>copyout()</code> performed after release?
<code>kernel/syscall.tbl</code>	What is the single source of truth for the ABI?
<code>tools/gensyscalls.py</code>	Which three files are generated and what appears in each?
<code>kernel/trap.c</code> , <code>usertrap()</code>	What does the kernel do when <code>scause == 8</code> ?
<code>kernel/syscall.c</code> , <code>syscall()</code> and <code>sys_sleep()</code>	Where are <code>a7</code> and <code>a0</code> interpreted?

Report Q1 - Reading map

In a compact table, identify the role of `user/usys.S`, `kernel/syscallnums.h`, `kernel/syscalltab.h`, and `kernel/syscall.tbl`. State which file may be edited and which three are generated.

3 Task 1 - Implement sleep

Open the supplied `user/sleep.c` and complete TODO regions S1–S3. The program is already included in the build.

LUIT GUEST SHELL - run only after the `luit$` prompt appears

`ticks` is a non-negative decimal integer. A successful call prints nothing, invokes the Luit `sleep()` system call, and exits with status 0.

Situation	Required action
Exactly one valid argument	Call <code>sleep(ticks)</code> once; print nothing on success; <code>exit(0)</code> .
No argument or more than one argument	Print <code>sleep: usage: sleep ticks</code> on descriptor 2; <code>exit(1)</code> .
Empty, negative, signed, or partly numeric input	Print <code>sleep: ticks must be a non-negative decimal integer</code> on descriptor 2; <code>exit(1)</code> .
The system call returns a negative value	Print <code>sleep: system call failed</code> on descriptor 2; <code>exit(1)</code> .
<code>ticks == 0</code>	Return immediately and successfully.

Ticks are not seconds

Pass the integer directly to the Luit `sleep()` system call. Do not multiply it by a guessed timer frequency and do not implement a busy-wait loop with `uptime()`.

After rebuilding and booting:

[illegible]

Report Q2 - Argument validation and syscall semantics

Explain why using `atoi(argv[1])` alone is insufficient. State what would happen in the baseline kernel if a negative value reached `sys_sleep()`, which compares elapsed ticks with a value converted to an unsigned type.

4 Task 2 - Implement pingpong

Open the supplied `user/pingpong.c` and complete TODO regions P1–P4. The pipe arrays and overall process structure are provided; the protocol and resource ownership are yours to implement.

Required contract: Required protocol

The program accepts no arguments. The parent and child communicate through **two unidirectional pipes**:

1. The parent sends exactly one byte to the child.
2. The child verifies that it received that byte and prints its PID followed by : `received ping`.
3. The child sends exactly one byte back to the parent.
4. The parent verifies that byte and prints its PID followed by : `received pong`.
5. The parent waits for the child before exiting.

A normal run must have exactly this two-line shape; PID values will differ:

EXPECTED OUTPUT / DISPLAY - do not type this block

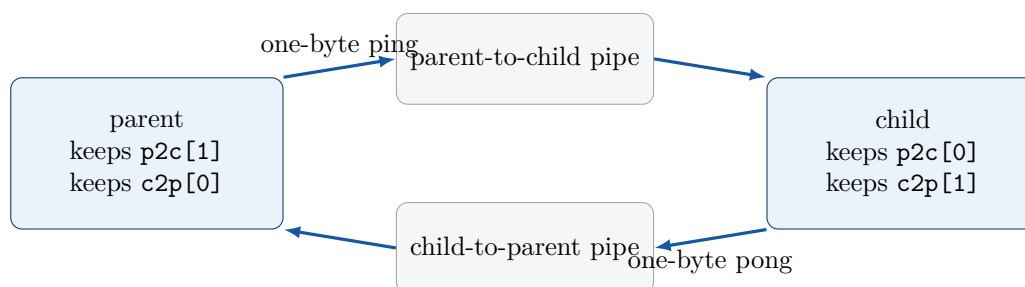
```
5: received ping
4: received pong
```

4.1 Descriptor ownership

Suppose the program creates arrays `p2c[2]` and `c2p[2]` before `fork()`. Immediately after the fork, both processes inherit all four descriptors. Each process must close the ends it will never use before beginning the exchange.

Before coding TODO P3 and P4, complete this ownership worksheet in your report:

Process	Descriptor	Direction	Keep/close	Reason
Parent	<code>p2c[0]</code>			
Parent	<code>p2c[1]</code>			
Parent	<code>c2p[0]</code>			
Parent	<code>c2p[1]</code>			
Child	<code>p2c[0]</code>			
Child	<code>p2c[1]</code>			
Child	<code>c2p[0]</code>			
Child	<code>c2p[1]</code>			



4.2 Correctness requirements

- Check the return value of both `pipe()` calls and `fork()`.
- Treat a `read()` or `write()` result other than 1 as an error.
- Verify the byte received; do not print success after receiving arbitrary data.
- Print errors to descriptor 2 and exit nonzero on failure.
- Close each retained descriptor as soon as its communication direction is finished.
- The child exits after sending the reply. The parent calls `wait(0)` and checks that a child was reaped.
- Do not use `sleep()` to control ordering. The protocol itself determines the output order.
- Do not use one pipe in both directions. Although a pipe object has read and write ends, this assignment requires two clearly directed channels.

Why the output order is deterministic

The child cannot print `received ping` until its blocking `read()` obtains the parent's byte. The parent does not print `received pong` until its own blocking `read()` obtains the child's reply, which is written after the child prints the first line. No timing delay is needed.

4.3 Required tests

Run the program under both one and four virtual harts:

LINUX HOST TERMINAL - run outside QEMU

```
make qemu CPUS=1
```

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```
pingpong
pingpong
pingpong
```

Exit QEMU, then repeat with:

LINUX HOST TERMINAL - run outside QEMU

```
make qemu CPUS=4
```

The two-line ordering must remain correct in every run.

Report Q3 - Pipe ownership

Draw the descriptor table immediately after `fork()` and again after both processes close their unused ends. Explain why keeping an extra write end open can prevent a pipe reader from observing end-of-file, even when the process that produced the intended data has exited.

Report Q4 - Synchronisation without delays

Explain the happens-before relationships that force the child line to appear before the parent line. State why adding `sleep()` would hide a design error rather than fix it.

5 Task 3 - Implement pstree

Open the supplied `user/pstree.c` and complete TODO regions T1–T7. This is the main Luit-specific user-space task.

5.1 The interface supplied by the release

`procstat()` has the declaration:

```
int procstat(struct pstat *records, int max_records);
```

It copies at most `max_records` in-use process records into the caller's array and returns the number copied, or -1 on failure. The maximum process-table size is 64.

The snapshot is intentionally not a global transaction. Each slot is copied safely, but a process may fork, exit, or be reparented while the scan proceeds. Your program must never crash or recurse forever because the snapshot changed.

5.2 Command-line and output contract

Required contract: Command-line interface

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```
pstree
pstree root-pid
```

With no argument, print the tree rooted at PID 1. With one positive decimal PID, print only that process and its descendants. Any other argument form prints `pstree: usage: pstree [root-pid]` to descriptor 2 and exits 1.

Each process is printed on one line using this format:

EXPECTED OUTPUT / DISPLAY - do not type this block

```
name(pid) [state]
```

Children are indented by two spaces per generation. Siblings must appear in increasing PID order. Use the same state names as `ps`: `used`, `sleep`, `runble`, `run`, `zombie`, and `?` for an unexpected value.

An example snapshot may look like this; exact PIDs and states are timing-dependent:

EXPECTED OUTPUT / DISPLAY - do not type this block

```
init(1) [sleep]
  sh(2) [sleep]
    pstree(12) [run]
  vdemo(7) [sleep]
    vdemo(8) [sleep]
    vdemo(9) [sleep]
    vdemo(10) [sleep]
```

5.3 Algorithmic requirements

1. Allocate a **static** fixed array of 64 `struct pstat` records. Luit gives a new program one 4 KiB user-stack page, so do not place several large working arrays on the stack.
2. Call `procstat()` exactly once. The output must represent one snapshot, not a mixture of repeated queries at different times.

3. Validate the requested root PID and locate it in the returned records.
4. Determine parent/child relationships using `ppid`; do not infer relationships from PID order or process names.
5. Sort siblings by numeric PID before printing.
6. Track which records have already been visited. Mark a record before descending to its children. If a changing or malformed snapshot creates a repeated edge or cycle, skip that edge and issue a concise warning on descriptor 2; never print a record twice.
7. Independently detect a broken parent link: a record has a positive `ppid`, but no returned record has that PID. Continue safely and, after printing, report `pstree: snapshot changed; N parent link(s) missing` on descriptor 2. Do not print unrelated unvisited records merely to make them appear in the selected subtree.
8. If the requested root does not exist, print `pstree: pid N not found` to descriptor 2 and exit 1.
9. Do not add a new syscall and do not read kernel memory directly.

You may use recursion because the process table contains at most 64 entries, or you may use an explicit traversal stack. Keep the snapshot and auxiliary arrays static or global; dynamic allocation is not required.

A snapshot, not the live kernel data structure

The array belongs to your process after `procstat()` returns. Traverse that local array without locks. Never assume its original order is meaningful: process-table slot order, PID order, and tree order are different concepts.

5.4 Required tests

First test the ordinary system:

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```
pstree
pstree 1
pstree 99999
pstree 1 2
```

Then create a visible multi-generation tree. Start a long-running `vdemo` in the background and immediately inspect:

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```
vdemo 4 100 > vdemo.log &
pstree
ps
```

Because the shell's background helper exits, the running `vdemo` may be reparented to `init`; this is correct and is itself an observation about orphan adoption.

Finally, run `pstree` repeatedly while short-lived processes are being created. The output may differ between snapshots, but the program must not hang, access invalid memory, duplicate a process, or recurse indefinitely.

Report Q5 - Constructing the tree

Explain why sorting the records returned by `procstat()` by PID is not sufficient to create a tree. Describe your representation for parent/child relationships, sibling ordering, and visited tracking. Include one actual `ps-tree` output with at least two generations below `init`.

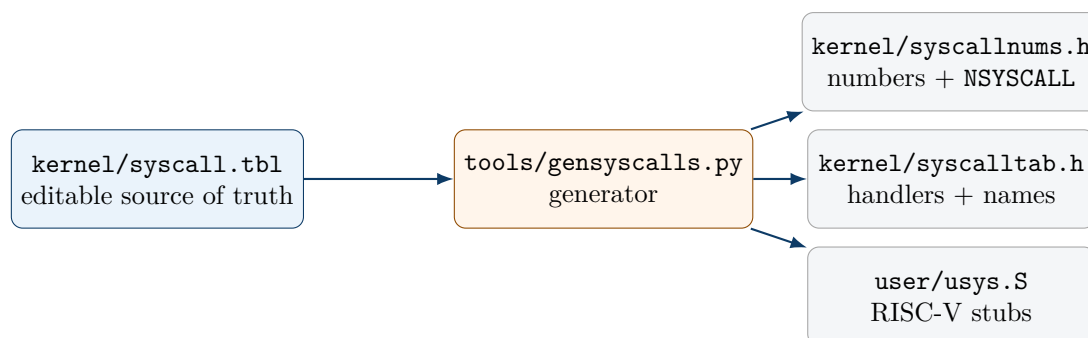
Report Q6 - Snapshot consistency

Give one concrete sequence of events in which a parent or child changes while `procstat()` is scanning the table. Explain how your program remains safe even though the resulting snapshot may not describe one exact instant globally.

6 Task 4 - Implement `tools/syscallmap.py`

This task runs on the Linux **Host**, not inside Luit. Complete TODO regions A1–A6 in the supplied `tools/syscallmap.py`; use only the Python 3 standard library.

6.1 The generated ABI pipeline



`make` runs the generator when required. Your checker must independently read the source table and all three generated files, report the complete map, and detect disagreement.

6.2 Required invocation and output

From the repository root:

LINUX HOST TERMINAL - run outside QEMU

```
python3 tools/syscallmap.py
```

For a valid tree, print a table with one row for every number from 1 through `NSYSCALL - 1`. Existing calls show their name and handler; unused numbers show `<unused>`. A compact example is:

EXPECTED OUTPUT / DISPLAY - do not type this block

NUM	NAME	HANDLER	USER-STUB
1	fork	sys_fork	yes
2	exit	sys_exit	yes
...			
17	<unused>	-	-
...			
23	procstat	sys_procstat	yes

syscallmap: ABI valid (22 calls, highest number 23, 1 gap)

Do not hardcode the count, the highest number, the gap at 17, or any syscall name. The output must adapt if the table changes.

6.3 Required validations

Your checker must detect and report all of the following before returning a nonzero host exit status:

1. a malformed non-comment row in `kernel/syscall.tbl`;
2. a duplicate syscall number;
3. a duplicate syscall name;
4. a non-positive syscall number;
5. a missing or incorrect `#define SYS_name number` in `kernel/syscallnums.h`;
6. an incorrect `NSYSCALL`; it must be one greater than the highest assigned number;
7. a missing or mismatched kernel handler entry in `kernel/syscalltab.h`;
8. a missing or mismatched syscall-name entry in `kernel/syscalltab.h`;
9. a missing or mismatched `SYSCALL name, number` stub in `user/usys.S`; and
10. any extra generated number, dispatch entry, name entry, or user stub that is not present in `kernel/syscall.tbl`.

If a generated file is missing, print a clear message directing the user to run `make`; do not produce a Python traceback for an expected input problem.

6.4 Fault testing your checker

First confirm a valid result:

LINUX HOST TERMINAL - run outside QEMU

```
make
python3 tools/syscallmap.py
echo $?
```

Then temporarily damage one generated line, run your checker, and immediately regenerate the file. For example, work on `user/usys.S`, which is generated:

LINUX HOST TERMINAL - run outside QEMU

```
cp user/usys.S /tmp/usys.S.lab01.backup
# Make one deliberate local edit to a SYSCALL number in user/usys.S.
python3 tools/syscallmap.py
echo $?
python3 tools/gensyscalls.py
python3 tools/syscallmap.py
```

The damaged version must be rejected with a nonzero exit status; the regenerated version must pass. Never commit or submit the deliberately corrupted generated file.

Report Q7 - One source of truth

Explain why generating both the user stub and kernel dispatch information from the same table prevents a class of ABI mismatch. Describe the deliberate fault you introduced, the diagnostic produced by your checker, and how regeneration repaired it.

7 Guided GDB Exercise - Trace sleep 5

This exercise is part of the assignment. It connects your user program to the RISC-V ABI and kernel control flow.

7.1 Inspect the generated user stub statically

Build first, then disassemble only the `sleep` stub:

LINUX HOST TERMINAL - run outside QEMU

```
make
riscv64-unknown-elf-objdump -d user/sleep | sed -n '/<sleep>:/,+6p'
```

You should find the three logical operations generated from `syscall.tbl`: load the syscall number into `a7`, execute `ecall`, and return. The C argument `ticks` is already in `a0` under the RISC-V calling convention.

7.2 Start the two-terminal debugging session

Terminal A:

LINUX HOST TERMINAL - run outside QEMU

```
make qemu-gdb CPUS=1
```

Terminal B, from the same repository directory:

LINUX HOST TERMINAL - run outside QEMU

```
gdb-multiarch kernel.elf
```

The repository's `.gdbinit` should connect to QEMU and print the available helpers. At the GDB prompt, allow Luit to boot:

GDB TERMINAL - commands entered at the (gdb) prompt

```
continue
```

Wait until the `luit$` prompt appears in Terminal A. Return to Terminal B and press **Ctrl-C** once to pause the emulated hart. Set a conditional breakpoint that selects an environment call made by a process whose name is exactly `sleep`:

GDB TERMINAL - commands entered at the (gdb) prompt

```
break usertrap if $scause == 8 && cpus[$tp].proc != 0 && cpus[$tp].proc->name[0] == 115 &&
    cpus[$tp].proc->name[1] == 108 && cpus[$tp].proc->name[2] == 101 && cpus[$tp].proc->
    name[3] == 101 && cpus[$tp].proc->name[4] == 112 && cpus[$tp].proc->name[5] == 0
continue
```

The numbers 115, 108, 101, 101, and 112 are the ASCII values for `s`, `l`, `e`, `e`, and `p`. This filter avoids stopping on the system calls used by `init` and the shell.

In Terminal A, enter:

LUIT GUEST SHELL - run only after the luit\$ prompt appears

```
sleep 5
```

GDB should stop at the entry to `usertrap()` for the `sleep` process.

7.3 Follow the same process across the boundary

At `usertrap()`, the live trap CSRs still describe the user-mode `ecall`. Save the current process and the original exception PC:

GDB TERMINAL - commands entered at the (gdb) prompt

```
set $p = cpus[$tp].proc
set $labpid = $p->pid
set $ecallpc = $sepc
p $p->pid
p $p->name
p $p->tf->a7
p $p->tf->a0
p/x $scause
p/x $sepc
trapregs
```

Expected interpretations:

- `a7` is 13 because `sleep` is syscall 13 in the supplied ABI.
- `a0` is 5 because the first integer argument uses `a0`.
- `scause` is 8, meaning an environment call from user mode.
- `sepc` identifies the address of the user-mode `ecall` instruction.

Now stop at `syscall()` for the same process:

GDB TERMINAL - commands entered at the (gdb) prompt

```
break syscall if cpus[$tp].proc != 0 && cpus[$tp].proc->pid == $labpid
continue
p $p->tf->a7
p $p->tf->a0
p/x $p->tf->epc
p/x ($p->tf->epc - $ecallpc)
```

The last expression should be 4: `usertrap()` has advanced the saved return PC past the four-byte `ecall` before dispatching the system call.

Continue to the handler and inspect the call chain:

GDB TERMINAL - commands entered at the (gdb) prompt

```
break sys_sleep if cpus[$tp].proc != 0 && cpus[$tp].proc->pid == $labpid
continue
bt
```

The backtrace should include `sys_sleep`, `syscall`, and `usertrap`. Do not use the live `scause` CSR here as the authoritative original cause: `usertrap()` enables interrupts before dispatch, so a device interrupt could have changed that CSR after entry. The earlier breakpoint captured it at the correct point.

Finally, stop when this process is about to return to user mode:

GDB TERMINAL - commands entered at the (gdb) prompt

```
break usertrapret if cpus[$tp].proc != 0 && cpus[$tp].proc->pid == $labpid
continue
set $p = cpus[$tp].proc
p $p->tf->a0
p/x $p->tf->epc
delete
```

`continue`

The `delete` command removes the temporary exercise breakpoints so that the program can issue its final `exit()` call without stopping again. The successful system-call return value in `a0` should be 0, and the saved `epc` should still identify the instruction immediately after `ecall`. Continuing allows the shell prompt to return.

If GDB stops repeatedly during boot

Press Ctrl-C, use `info breakpoints`, and delete unintended broad breakpoints. An unconditional breakpoint on `usertrap` stops for every user system call; retain the process-name and `scause` condition shown above.

Report Q8 - System-call path

Provide the disassembled `sleep` stub and a compact GDB transcript showing the process name/PID, `a7`, `a0`, the live `scause` and `sepc` at `usertrap()`, the four-byte advancement of the trapframe `epc`, the handler backtrace, and the return value. Then explain, in your own words, the complete path from `sleep.c` through the generated user stub, `ecall`, `usertrap()`, `syscall()`, and `sys_sleep()`, ending at `usertrapret()`.

8 Integration and Testing Requirements

8.1 Required source-tree changes

Your implementation changes are restricted to:

- `user/sleep.c`;
- `user/pingpong.c`;
- `user/pstree.c`; and
- `tools/syscallmap.py`.

You also complete the evidence templates under `lab01/`. Do not modify the Makefile, files under `kernel/`, `hal/`, or `mkfs/`, the public tests, the packaging tool, or generated files `user/usys.S`, `kernel/syscallnums.h`, and `kernel/syscalltab.h`.

8.2 Build and regression sequence

Before submission, run:

LINUX HOST TERMINAL - run outside QEMU

```
make clean
make
make lab01-check
python3 tools/syscallmap.py
make grade
make -C lab01
```

Then manually test the new guest programs under at least `CPUS=1` and `CPUS=4`.

8.3 Minimum test matrix

Component	Required cases	Evidence of correctness
<code>sleep</code>	0, small positive value, missing arg, negative, mixed text, overflow, extra arg	no output on success; exact errors; no hang
<code>pingpong</code>	repeated runs at 1 and 4 harts	exactly two ordered lines; process exits; shell returns
<code>pstree</code>	default root, explicit root, absent root, invalid args, active <code>vdemo</code>	hierarchy by <code>ppid</code> ; sorted siblings; no duplicate or infinite traversal
<code>syscallmap.py</code>	valid generated files, one deliberately mismatched stub, regenerated files	exit 0 only for consistent ABI; clear mismatch diagnostic
Baseline	<code>make grade</code>	all public regression tests pass

8.4 What grading tests will examine

The grading environment may vary the hart count and process timing. Tests will check the documented contract, including malformed arguments, exact success/error channels, descriptor cleanup, repeated execution, arbitrary process-record order, absent or changing parent records, and a temporarily modified syscall table or generated file. A solution that hardcodes today's PIDs, syscall count, gap, or names will fail.

9 Submission Through the OneDrive File-Request Link

9.1 Complete the identity and evidence files

Before packaging, complete:

- `lab01/README.txt`: name, exact nine-digit roll number, laboratory section, and IITG email;
- `lab01/report.tex`; the packaging tool rebuilds `lab01/report.pdf` from this source;
- `lab01/gdb-trace.txt`: name, exact nine-digit roll number, and the required trace evidence; and
- `lab01/LLM_LOG.md`: name, exact nine-digit roll number, and the exact no-use statement when applicable.

9.2 Create the roll-numbered ZIP automatically

From the repository root, replace the example with your own nine-digit roll number:

LINUX HOST TERMINAL - run outside QEMU

```
python3 tools/package_lab01.py 230101037
```

The tool validates the required files, checks that the roll number appears in all identity-bearing evidence files, rejects remaining template and starter implementation markers, verifies the protected release files, rebuilds the report from `report.tex`, creates the ZIP, reopens it, and verifies its contents. The result must be named exactly:

EXPECTED OUTPUT / DISPLAY - do not type this block

```
230101037_Lab01.zip
```

and must contain exactly one top-level folder:

EXPECTED OUTPUT / DISPLAY - do not type this block

```
230101037_Lab01/  
|-- user/  
|   |-- sleep.c  
|   |-- pingpong.c  
|   '-- pstree.c  
|-- tools/  
|   '-- syscallmap.py  
|-- report.pdf  
|-- gdb-trace.txt  
|-- README.txt  
'-- LLM_LOG.md
```

Do not rename files inside the generated archive. Do not upload your complete Luit repository, build products, object files, `fs.img`, generated syscall files, or a loose folder.

9.3 Upload procedure

1. Open the official **Lab 1 OneDrive file-request link** announced by the TA.
2. Sign in with your IITG Microsoft account when prompted.
3. Upload exactly one file: `ROLLNUMBER_Lab01.zip`.
4. Wait for the successful-upload confirmation before closing the browser.
5. Retain your local ZIP and the confirmation until marks are released.

The submission deadline will be announced in the lab. Complete the upload before that announced deadline. You may upload from any suitable computer and network, and the upload page will not show other students' work.

Avoid repeated uploads. If a correction is unavoidable, upload a newly verified ZIP with the same required roll-number name before the announced deadline. The latest valid submission received before closure will be evaluated. A ZIP with another student's roll number, a malformed roll number, or no roll number in `README.txt` is not a valid submission.

Do not create a shared roll-number folder in OneDrive

Students do not receive edit access to the collection folder. Submit the one ZIP generated by `package_lab01.py`. The ZIP itself contains the required roll-numbered top-level folder, which allows the TA collection script to copy and evaluate submissions automatically.

9.4 Final submission checklist

Check before upload

- ☐ `make clean && make` succeeds without errors.
 - ☐ `make lab01-check` passes and no protected file was changed.
 - ☐ `sleep` passes valid, malformed, overflow, and argument-count tests.
 - ☐ `pingpong` gives exactly two ordered lines at `CPUS=1` and `CPUS=4`.
 - ☐ `pstree` uses one snapshot, sorted siblings, visited tracking, and safe root handling.
 - ☐ `syscallmap.py` passes valid files and rejects the deliberate mismatch.
 - ☐ `make grade` passes.
 - ☐ `lab01/report.tex`, `gdb-trace.txt`, `README.txt`, and `LLM_LOG.md` are complete.
 - ☐ The package command reports "Created and verified".
 - ☐ The uploaded file is exactly `YOURROLL_Lab01.zip`.
-

10 Assessment Rubric

Component	Marks	What earns full credit
Build and integration	10	warning-free clean build; starter-check and baseline regression pass; no protected-file modifications
<code>sleep</code>	10	exact interface, complete validation, correct syscall use, error handling
<code>pingpong</code>	20	two-pipe protocol, correct descriptors, deterministic ordering, checked I/O, cleanup, wait
<code>pstree</code>	25	one safe snapshot, correct hierarchy, deterministic sibling order, root handling, cycle/instability safety
<code>syscallmap.py</code>	15	no hardcoding, all required consistency checks, useful diagnostics and host exit status
GDB evidence and written report	10	accurate ABI values, complete path, concise evidence, technically correct reasoning
Viva	10	ability to explain and modify your own design without external assistance
Total	100	

A program that produces one expected sample output but violates the stated interface, leaks descriptors, depends on timing, hardcodes the ABI, or cannot be explained in the viva will not receive full functional credit.

11 AI/LLM Use and Academic Integrity

The course-wide policy in `docs/LLM_POLICY.md` applies.

Mode	Examples for this lab
GREEN	explanations of supplied Luit code; RISC-V ABI questions; explanations of pipe EOF; help interpreting your compiler error or GDB output; grammar improvement for your report
AMBER	brainstorming tests; reviewing a diff you already wrote; discussing trade-offs in your own <code>pstree</code> representation; must be logged
RED	generating solution code or line-by-line solution pseudocode for <code>sleep.c</code> , <code>pingpong.c</code> , <code>pstree.c</code> , or <code>syscallmap.py</code> ; agentic editing of your working tree; using another student's or a public solution's implementation

For every AMBER interaction, record: date, tool/model, what you asked, what you used, where it affected the work, and how you verified it. An empty truthful log is acceptable; a hidden or false log is an academic-integrity violation.

12 Getting Unstuck

Problem	Action
A fresh release does not build or grade	Stop and show the complete output to a TA. Do not patch the kernel to hide an environment/release problem.
New command says <code>sh: cannot run sleep</code>	Run <code>make lab01-check</code> . If the supplied program is absent from the starter Makefile, stop and contact a TA; otherwise rebuild <code>fs.img</code> .
<code>sleep -1</code> appears to hang	Your user program passed a negative value to <code>sys_sleep</code> ; validate before the syscall. Exit QEMU and fix the parser.
<code>pingpong</code> hangs	Draw all eight post-fork descriptor copies. Check the kept direction in each process and close every unused end before communication.
Parent prints before child	The protocol is wrong or the parent prints before reading the reply. Do not add a delay.
<code>pstree</code> loops or duplicates rows	Add a visited array indexed by snapshot record, and mark a record before traversing its children.
<code>pstree</code> does not resemble <code>ps</code> order	That is expected. <code>ps</code> prints snapshot order; a tree must follow <code>ppid</code> and sort siblings.
<code>syscallmap.py</code> cannot find generated files	Run <code>make</code> from the repository root. Use paths relative to the root and produce a clear diagnostic.
GDB cannot auto-load <code>.gdbinit</code>	Follow GDB's printed <code>add-auto-load-safe-path</code> instruction, then restart the two-terminal session.
GDB stops on many system calls	Delete broad breakpoints and use <code>break sys_sleep</code> .
QEMU will not quit	Press Ctrl-A, release, then x.

13 Viva Preparation

You should be able to answer the following without opening an AI assistant or searching the web:

1. Why can a negative `sleep` argument become an extremely large wait in the kernel?
2. Which descriptors exist in each `pingpong` process immediately after `fork()`?
3. Why are two pipes used? What would be ambiguous or unsafe in a poorly designed one-pipe exchange?
4. What exact event causes a pipe read to return 0?
5. Why can `procstat()` be memory-safe but not globally instantaneous?
6. Why does `pstree` need `ppid`; why cannot it derive the hierarchy from PID values?
7. What protects a process slot while the kernel copies it into a temporary `struct pstat`?
8. Why must the user program traverse its copied array without kernel locks?
9. How does `kernel/syscall.tbl` reach both `a7` in user space and the dispatch function in the kernel?
10. Why is syscall number 17 displayed as unused rather than treated as an error?
11. At `sys_sleep()`, what do `a7`, `a0`, `scause`, and `epc` mean?
12. Why does `usertrap()` advance the saved `epc` by 4 after an `ecall`?
13. Where is the system-call return value stored before `sret` returns to user mode?
14. Which part of your implementation would fail first if the process table contained 64 active records?

14 The Main Ideas to Carry Forward

Idea	What you should now understand
Process creation	<code>fork()</code> creates a new execution context that inherits descriptors and memory contents, while receiving a distinct PID and return value.
Pipe communication	A pipe is a kernel byte channel; correctness depends on direction, descriptor ownership, blocking, and complete closure.
Process observation	A user tool sees a copied snapshot through a system-call interface, not the kernel's live pointers.
Deterministic presentation	Kernel snapshot order is not a user interface; tools impose sorting and hierarchy deliberately.
Generated ABI	One declarative table can keep syscall numbers, kernel dispatch, names, and user assembly stubs consistent.
RISC-V boundary	Arguments arrive in <code>a0-a5</code> , the syscall number in <code>a7</code> , <code>ecall</code> raises cause 8, and the result returns in <code>a0</code> .
Engineering discipline	Validate input, check every system-call result, close owned resources, test under different hart counts, and preserve regressions.

Build it, observe it, explain it.

A correct operating-system program is not merely one that works once. Its process relationships, descriptor ownership, ABI assumptions, and failure behaviour must remain correct under different schedules.